MAGNET · TAMAN SAFARI

Zoo Vision Project guide

How the AI video analytics backend fits together: what each part does, how data moves between them, the data model, the API, and how to run it.

Contents

1. What Zoo Vision is
2. The system at a glance
3. The parts, one by one
4. How data moves
5. Data model
6. Database structure
7. Use cases
8. Users, roles and login
9. Connecting the frontend
10. API reference
11. WebSocket and engine channels
12. Running it
13. Repo map and where to read more
14. Glossary

Repository	`Aryaputra34/zoo-vision` (backend); dashboard in `zoo-vision-fe`
Written from	`main` at commit `6b392b0` (Phase 4b merged), 27 October 2026
Sources	The production design spec, the READMEs, `docs/fe-contract.md`, `apps/api/openapi.json` and `packages/contracts`
When in doubt	The code and `openapi.json` are right; this guide is a summary of them

1. What Zoo Vision is

Zoo Vision is AI video analytics for one park, Taman Safari. About **12 cameras** watch cashier desks, the restaurant (occupancy and tables), the vehicle gate and the pony rides. A Python vision engine runs YOLO detection and ByteTrack tracking on each camera. It turns what it sees into **events** (a vehicle crossed the gate), **intervals** (a desk was unattended from 10:02 to 10:09) and **metrics** (restaurant occupancy each minute). A NestJS API stores them in Postgres and serves them to a web dashboard, along with live video, snapshots and clips.

Ground rules of the design

- **One park, one on-site server.** No cloud, no multi-site, no tenants. Everything runs in one Docker Compose stack on a GPU server at the park.
- **The database is the source of truth for config.** An admin adds cameras and draws zones in the web UI. A change applies live, between two frames, with no stream drop and no restart of other cameras.
- **Use cases will change and grow.** Adding one must not need a database migration. Each use case is defined once, as JSON Schemas in `packages/contracts`, and the TypeScript and Python types are generated from them.
- **Nx Witness is optional.** It was the centre of the prototype. Now MediaMTX records the video and Nx is kept only as a possible future integration (`integrations/nx/plugin/`, frozen).
- **Small team.** One backend/CV developer owns this repo; a frontend team builds the dashboard against a versioned contract.

Where the project stands

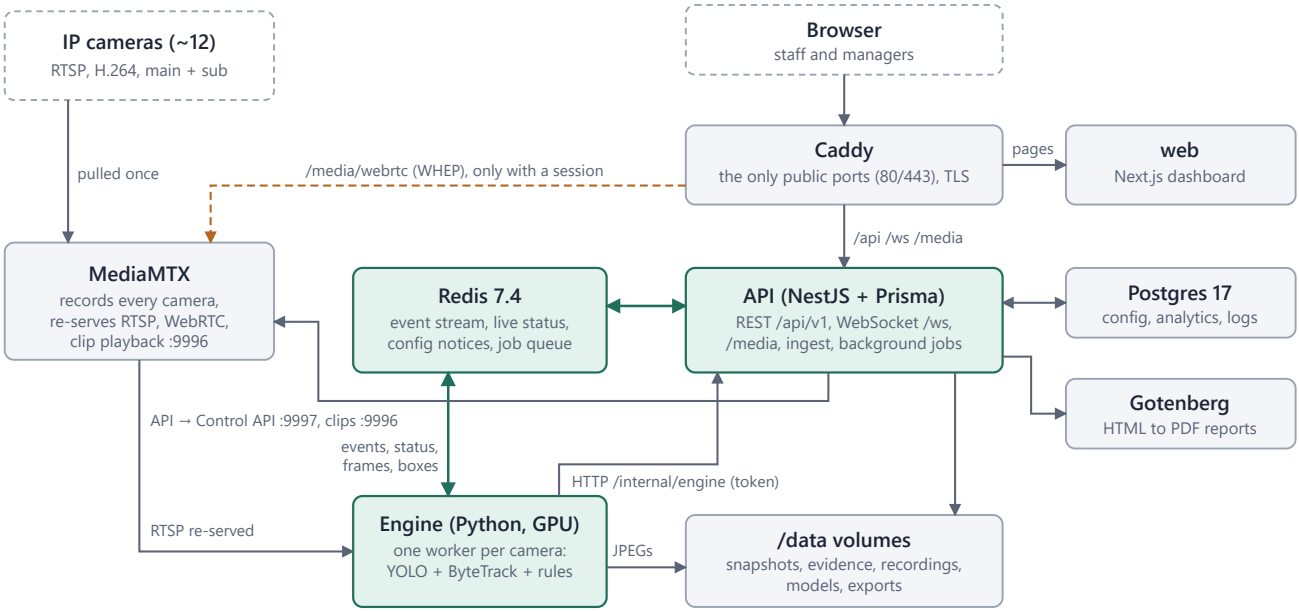
The rebuild from the prototype is split into phases, each ending in a working system. Each phase has a plan in `docs/superpowers/plans/`, and the larger ones have a design spec in `docs/superpowers/specs/`.

Phase	What it delivered	Status
0 Restructure	Monorepo layout (`services/engine`, `apps/api`, `packages/contracts`, `deploy`), binaries out of git, CI	done
1 Backbone	Postgres + Prisma, Redis stream, contracts package, auth (sessions, API tokens), users, audit, cameras, event ingest	done
2 Live config	Engine supervisor, use case registry, typed settings applied live, shared model pool, status heartbeats, AI model registry	done
3 Web on the API	Dashboard moved onto the API, Caddy as the single origin, WebSocket live updates, reports, lists, CSV exports, system log	done
4a Video	MediaMTX path sync from the API, WebRTC live view with boxes drawn in the browser, superadmin debug view	done
4b Evidence	Kept clips, retention jobs, disk warnings, site settings, PDF reports through Gotenberg	done
5 Hardening	Backups, install and update scripts, offline update path, system status page, 24 h soak test, ADR-009	next

Planned later, each with its own spec: fraud detection (matching camera counts against POS and ticket sales) and an Nx bridge. Some hooks for fraud detection already exist: the analytics tables are append-only, and config changes on audit cameras are flagged.

2. The system at a glance

Eight services in one Compose stack, plus the browser. The engine and the API never import each other's code. At runtime they talk through Redis, and the engine also fetches its camera list from the API over HTTP.



Green: the backend core. Dashed boxes: outside the stack. The dashed orange line is live video, which goes from MediaMTX to the browser without passing through the API; Caddy asks the API whether the browser is logged in first.

The four paths through the system

Path	Route
Analytics	Camera → MediaMTX → engine → Redis stream `events` → API ingest → Postgres → REST and WebSocket → dashboard
Config	Dashboard → API saves a new config version → Redis `cfg:changed` → engine fetches from `/internal/engine/cameras` → worker applies it between two frames → status shows the applied version
Live video	Browser → Caddy `/media/webrtc` (after `forward_auth` to `GET /api/v1/auth/check`) → MediaMTX WHEP. Boxes come separately: engine → Redis `cam:{id}:boxes` → API → WebSocket room `camera:{id}` → canvas over the video
Evidence	Engine writes the event snapshot to `/data/snapshots`; MediaMTX records continuously; the API's pin job copies a clip window from MediaMTX playback into `/data/evidence` for events worth keeping

3. The parts, one by one

Engine: `services/engine/` (Python 3.12, uv)

One process holds a **supervisor** and **one worker thread per camera**. Each worker owns its stream (`core/stream_manager.py`) and its use case pipeline. The supervisor (`core/supervisor.py`) keeps the set of workers in line with the cameras the API says this engine should run.

- **Config comes from the API, not files.** At start, and on every `cfg:changed` message, Redis reconnect, or 60 s timer, the engine calls `GET /internal/engine/cameras?group=<ENGINE_GROUP>`. Its only static config is environment variables (`REDIS_URL`, `ENGINE_TOKEN`, `API_URL`, `DEVICE`, …).
- **Use cases are plug-ins.** Each is a folder `usecases/<id>/pipeline.py` holding one `BasePipeline` subclass. They are found by scanning the folder, so adding one doesn't mean editing a central list.
- **Shared model pool** (`core/model_pool.py`): one loaded model per (file, image size), shared by every camera that uses it, with an inference thread that can batch frames (`ENGINE_MAX_BATCH`).
- **Outputs:** stream messages through `core/event_stream.py` (built by `core/messages.py`), a status heartbeat every 5 s (`core/status.py`), a JPEG frame every 2 s, per-frame boxes, and event snapshots (`core/snapshot_store.py`).
- **Model jobs:** the engine also validates uploaded models and converts `.pt` to ONNX, taking jobs from the Redis list `model:jobs` (`core/model_jobs.py`).
- **Engine groups:** each camera has an `engineGroup`. A second engine process with another `ENGINE_GROUP` can take some cameras, for example a crash-prone one.

API: `apps/api/` (NestJS 11, Prisma 7)

Everything the browser and the engine need from the backend. Each folder in `src/` is a Nest module:

Modules	Responsibility
`auth`, `users`, `api-tokens`	Cookie sessions stored in Redis, the three roles, API tokens for scripts and Swagger
`cameras`, `usecases`	Cameras and their versioned config (save, history, rollback); serves the use case schemas the zone editor needs
`models`	The AI model registry: upload (ONNX only), rescan of the inbox folder, validate, convert, delete
`ingest`	Reads the Redis stream into Postgres in transactions, idempotently, and dead-letters bad entries
`engine-internal`	`/internal/engine/*`: the engine's routes, guarded by `ENGINE_TOKEN`, never routed by Caddy
`live`	socket.io gateway at `/ws`: new events, data-changed pings, camera status, boxes, audit notices
`reports`, `exports`	Stats and time series; streamed CSV; PDF report jobs rendered through Gotenberg
`media`, `mediamtx`, `evidence`	Serves snapshots and clips behind the session; keeps MediaMTX paths in sync with the cameras; keeps clips
`jobs`	Background jobs: camera offline check, evidence pins, retention, disk and free-space checks
`audit`, `system-log`, `site-settings`	Audit trail of every change and export, login history, system events; park name, time zone, retention days

Contracts: `packages/contracts/` (JSON Schema)

The **single source of truth** for the stream messages, the per-use-case settings and live values, and the WebSocket messages. `npm run generate` writes TypeScript types (for the API and one self-contained file for the frontend) and Pydantic models (into `services/engine/contracts/`). CI fails if the generated files are out of date, so the engine, API and dashboard can't drift apart.

Supporting services

MediaMTX

Pulls each camera once and records it. It re-serves the stream over RTSP to the engine, over WebRTC to browsers, and serves clip playback on :9996. The API adds and changes its paths at runtime through the Control API (:9997), so adding a camera in the UI needs no SSH.

Redis

The only channel between API and engine, plus session storage. AOF persistence means the event stream survives a Redis restart.

Postgres

Config, users, logs and the three analytics tables. The running API connects as `zoo_api`, which can't update or delete analytics rows; migrations run as the owner role `zoo_owner`.

Caddy

One origin for everything: `/` to the dashboard; `/api`, `/ws` and `/media` to the API; `/media/webrtc` to MediaMTX after an auth check; `/internal/*` answers 404. One origin means plain cookies and no CORS.

Gotenberg

Headless Chromium that turns the dashboard's print-styled report page into a PDF summary.

web (separate repo)

`zoo-vision-fe`, Next.js + Tailwind. UI only: it talks to the API from the browser and never touches the database.

4. How data moves

4.1 From a camera frame to a row in Postgres

1 Detect
The worker runs the model and tracker, then the use case rules (zones, tripwire, timers).

→

2 Snapshot
For an event, it draws boxes and zones on the frame and saves a JPEG to `/data/snapshots`.

→

3 XADD
It appends a contract-checked message to the Redis stream `events`. Once that returns, the event is durable.

→

4 Ingest
The API reads batches as consumer group `api`, writes them in one transaction (idempotent on message id), then ACKs.

→

5 Push
After the commit: `event.created` and `data.changed` over the WebSocket.

There are four message kinds: `event`, `interval_open`, `interval_close` (the two share a session id) and `metric`. Delivery is at-least-once with no double counting: if the API crashes mid-batch, it re-reads its unacknowledged entries on restart and the inserts are idempotent.

If this fails...	...this happens
API or Postgres down	Messages wait in the Redis stream and are processed when it comes back. The engine carries on.
Redis briefly down	The engine holds events in a small bounded memory buffer and reports how many it dropped (`events_dropped`).
A bad message	Invalid, or failed too many times: it goes to the `DeadEvent` table and is ACKed, so it never blocks the rest.
Engine restart	It sends `engine_started`; the API closes that group's open intervals with `endReason = engine_restart`.
Camera goes dark	Its status key expires after about 15 s; the API opens a `camera_offline` interval, so reports can tell "no rides" from "camera off".

4.2 A config change reaching a running camera

1 Edit
A manager draws zones on the camera's latest frame (`GET /cameras/:id/frame`).

→

2 Save
`POST /cameras/:id/config` with `baseVersion`. The API checks the schema and role rules and stores version N+1.

→

3 Notify
The API publishes `cfg:changed {cameraId, version}` on Redis.

→

4 Apply
The engine fetches its cameras, validates the settings with Pydantic and the worker applies them between two frames.

→

5 Confirm
Status reports `appliedVersion` = N+1, or a `configError` while it keeps running N.

- Every event carries the `configVersion` it was produced under, so a jump in a report can be traced to an edit.
- Rollback makes an older version active again by creating a new version; history is never rewritten.
- Two people editing at once: the second save gets `409 STALE_VERSION`.
- What a change costs: geometry and thresholds apply on the next frame and keep tracks and counts. Tracker tuning restarts track ids once. A new model loads in the background and then swaps in. A new source URL reconnects only that camera. A new use case rebuilds that camera's pipeline.

4.3 Live view and evidence

- **Video:** WebRTC (WHEP) straight from MediaMTX, H.264 passed through without re-encoding. Caddy lets the call through only after `GET /api/v1/auth/check` answers 204.
- **Boxes:** the browser joins the WebSocket room `camera:{id}`. The API subscribes to Redis `cam:{id}:boxes` only while someone is in that room, and the browser draws the boxes on a canvas over the video.
- **Use case overlays** (table badges, occupancy, desk state) are drawn by the dashboard from the camera's status, so the engine never encodes live video.
- **Debug view** (superadmin only): `/media/debug/:id` shows the engine's own annotated frames as MJPEG. The engine draws them only while someone is watching.
- **Clips:** `GET /media/clips/:eventId` serves a kept clip from `/data/evidence`, or cuts the window around the event from MediaMTX's recordings while they still exist. Clips are kept automatically for warning and critical events and every event on an audit camera, and by hand with `POST /events/:id/evidence`.

5. Data model

Postgres via Prisma (`apps/api/prisma/schema.prisma`). Times are stored in UTC; reports use the site time zone (Asia/Jakarta by default, a site setting).

Analytics: stored by shape, not by use case

Every use case produces some mix of three shapes, so three generic tables hold all of them. A new use case adds rows, not tables; its specific fields live in a `data` JSONB column checked against its contract.

Table	Shape	Key columns	Examples
`Event`	Something happened at a moment	time, camera, useCase, type, severity, configVersion, schemaVersion, snapshot path, `data`	vehicle crossing, pony departure, capacity alert
`Interval`	A state that lasted	camera, useCase, kind, subjectId, start, end, endReason, `data`	table session, desk unattended, camera offline
`Metric`	A number over time, one row per minute	camera, useCase, name, minute, value	occupancy, queue length

The pattern mirrors OpenTelemetry's logs, spans and metrics. Live state (current occupancy, fps) is never stored here; it lives in Redis `cam:{id}:status` .

Config, people and logs

A summary; every column, key and index is in section 6.

Table	What it holds
`Camera`	Slug id (`cam_gate_entry_01`), name, area, useCase, enabled, source type and encrypted stream URLs, `inferenceStream` (MAIN or SUB), target FPS, engineGroup, `auditCamera` , recording settings, optional `nxCameraId` , active config version
`CameraConfigVersion`	One row per saved config: the settings JSONB (zones, tripwires, tables with stable ids, thresholds), schemaVersion, author, time, note
`AiModel`	Registry entry: format, file, sha256, class names, image size, status `UPLOADED → VALIDATING → READY / FAILED` , validation report
`User` , `ApiToken`	Accounts with a role; hashed API tokens with a role, last use and revocation
`AuditLog` , `LoginLog`	Before/after of every change and every export; every login attempt and its result
`SiteSettings`	One row: park name, time zone, retention days
`ExportJob` , `DeadEvent`	PDF report jobs; stream entries that could not be stored

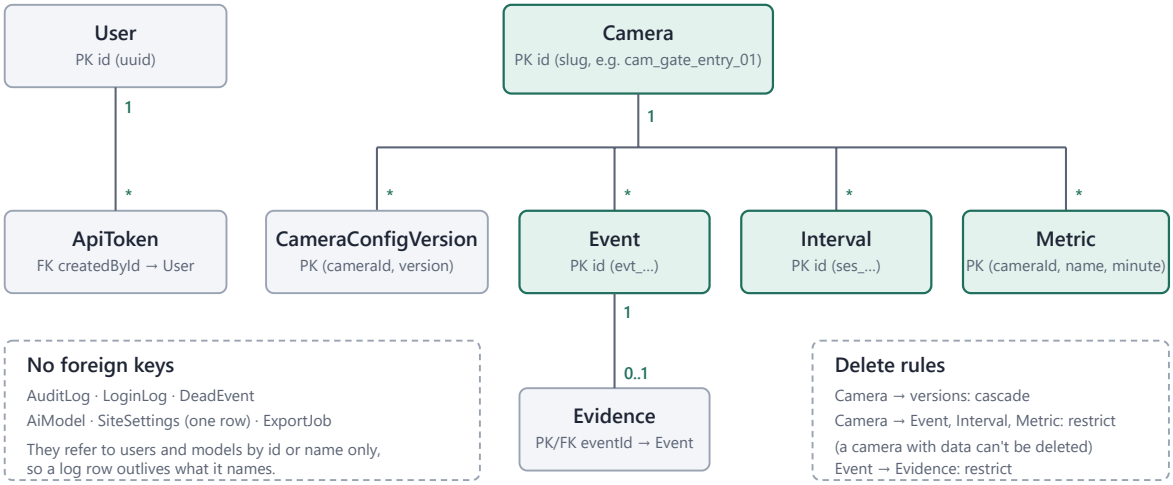
> **Append-only by design.** The API's database role can only SELECT and INSERT on `Event` , `AuditLog` , `LoginLog` and `DeadEvent` , and can update only specific columns of `Interval` and `Metric` . Counts can't be quietly edited, which the future fraud module relies on. **Every migration that adds a table must also add its `GRANT` s** (see `apps/api/README.md`).

Do I need a migration?

Usually not. A new field on a use case's events, intervals, metrics or settings is a **contract change** (`npm run generate` in `packages/contracts`), not a migration. A Prisma migration is for the structural tables, new tables and new indexes.

6. Database structure

Fourteen tables in Postgres 17, defined in `apps/api/prisma/schema.prisma`. Migrations are in `apps/api/prisma/migrations/`; some indexes, the grants and one check constraint exist only in the migration SQL. All times are `timestamptz(3)` in UTC.



Relations in the schema. Green: the camera and the three analytics tables. `Event.cameraId` is nullable, because system events such as `engine_started` belong to no camera.

Who may do what: grants for the API's role `zoo_api`

Tables	Allowed
`User`, `ApiToken`, `Camera`, `AiModel`, `Evidence`, `ExportJob`	SELECT, INSERT, UPDATE, DELETE
`CameraConfigVersion`	SELECT, INSERT, DELETE (no UPDATE: a version never changes)
`Event`, `AuditLog`, `LoginLog`, `DeadEvent`	SELECT, INSERT only (append-only)
`Interval`	SELECT, INSERT, and UPDATE of `end`, `endReason`, `data` only (closing an interval)
`Metric`	SELECT, INSERT, and UPDATE of `value` only (the minute's upsert)
`SiteSettings`	SELECT, UPDATE

Analytics tables

Column	Type	Meaning
`Event`: **something happened at a moment**		
`id`	text, PK	`evt_…`, made by the engine, so a redelivered message inserts nothing new
`time`	timestamptz	When it happened (engine clock)
`cameraId`	text, FK, nullable	The camera; null for system events
`useCase`, `type`	text	Which contract, which event type (`vehicle_crossing`, …)
`severity`	enum	`info`, `warning`, `critical`
`configVersion`	int, nullable	The camera config version that produced it

Column	Type	Meaning
`schemaVersion`	int	The use case schema version of `data`
`snapshotPath`	text, nullable	The JPEG under `/data/snapshots`
`data`	jsonb	The use case's fields, as its contract defines (plate, direction, ...)
`ingestedAt`	timestamptz	When the API stored it
Indexes: (cameraId, type, time), (time)		
`Interval` : **a state that lasted**		
`id`	text, PK	`ses_...`, the session id shared by the open and close messages
`cameraId`	text, FK	The camera
`useCase`, `kind`	text	Which contract; which kind (`table_session`, `desk_unattended`, `camera_offline`)
`subjectId`	text, nullable	What it is about inside the camera: a table id, a desk
`start`, `end`	timestamptz	`end` is null while it is still open
`endReason`	text, nullable	Why it closed, e.g. `engine_restart`
`configVersion`, `schemaVersion`, `data`	int, int, jsonb	As on `Event`
Indexes: (cameraId, kind, start); a partial unique index allows only one open `camera_offline` interval per camera		
`Metric` : **a number per minute**		
`cameraId`, `name`, `minute`	PK together	One row per camera, metric name and minute
`useCase`	text	Which contract
`value`	float	The value for that minute
`Evidence` : **a kept clip (also protects the event's snapshot from retention)**		
`eventId`	text, PK, FK	The event
`cameraId`, `eventTime`	text, timestamptz	Copied from the event for the job's queries
`reason`	enum	`SEVERITY`, `AUDIT_CAMERA`, `USER`
`status`	enum	`QUEUED`, `KEPT`, `NO_RECORDING`, `FAILED`
`runAt`, `attempts`, `error`	timestamptz, int, text	The pin job's retry state
`filePath`, `sizeBytes`	text, int	The clip under `/data/evidence`
Indexes: (status, runAt), (cameraId, eventTime)		

Cameras and config

Column	Type	Meaning
`Camera`		

Column	Type	Meaning
`id`	text, PK	Slug chosen at creation, e.g. `cam_gate_entry_01`
`name`, `area`	text	Display name; optional area
`useCase`	text	Which use case it runs
`enabled`	bool	Off: the engine stops it
`sourceType`	enum	`RTSP`, `MEDIAMTX`, `NX`, `FILE`
`mainStreamUrlEnc`, `subStreamUrlEnc`	text	Stream URLs, encrypted with `CAMERA_URL_KEY` (they contain passwords)
`inferenceStream`, `recordingStream`	enum	`MAIN` or `SUB`: which stream the engine analyses, which one MediaMTX records
`targetFps`	int	Frames per second the engine analyses (default 5)
`engineGroup`	text	Which engine process runs it (default `default`)
`auditCamera`	bool	Flag changes, notify managers, keep every event's clip
`recordingEnabled`	bool	Continuous recording on or off
`retentionDays`, `evidenceRetentionDays`	int, nullable	Overrides of the site defaults; null uses the site setting
`clipBeforeSec`, `clipAfterSec`	int	Clip window around an event (default 15 + 15)
`nxCameraId`	text, nullable	For a future Nx integration
`activeVersion`	int	The config version in force
`firstSeenAt`	timestamptz, nullable	First time the engine reported this camera
`createdAt`, `updatedAt`	timestamptz	
`CameraConfigVersion`: **one row per save**		
`cameraId`, `version`	PK together	Versions count up per camera; rollback adds a new one
`useCase`, `schemaVersion`	text, int	Which contract the settings follow
`settings`	jsonb	Zones, tripwires, tables and thresholds, checked against the use case's `settings.schema.json`
`authorType`, `authorId`, `authorName`	text	Who saved it (user or token)
`note`, `createdAt`	text, timestamptz	Optional comment; when
`AiModel`: **the model registry**		
`id`	uuid, PK	
`name`, `version`, `originalName`	text	Display name, optional version, the uploaded file's name
`format`	enum	`ONNX`, `PT`, `ENGINE`
`source`	enum	`BUILTIN`, `FILESYSTEM` (inbox), `UPLOAD`, `CONVERTED`
`status`	enum	`UPLOADED`, `VALIDATING`, `READY`, `FAILED`, `CONVERTING`

Column	Type	Meaning
`filePath`, `sha256`	text, unique	Path under the models folder; checksum (null only while converting)
`sizeBytes`, `classNames`, `validation`	bigint, jsonb, jsonb	Size; class list after validation; the validation report (ok, latency, device, error)
`convertedFromId`, `createdById`, `notes`	uuid, uuid, text	Source model of a conversion; who added it
`SiteSettings` : **exactly one row** (`CHECK id = 1`)		
`siteName`, `timeZone`	text	Park name; IANA zone for reports (default `Asia/Jakarta`)
`recordingRetentionDays`	int	Continuous recordings (default 7)
`snapshotRetentionDays`, `auditSnapshotRetentionDays`	int	Snapshots (default 90); snapshots of audit cameras (default 365)
`evidenceRetentionDays`	int	Kept clips (default 365)

People, logs and jobs

Column	Type	Meaning
`User`		
`id`	uuid, PK	
`username`	text, unique	Login name
`name`, `passwordHash`	text	Display name; bcrypt hash
`role`, `active`	enum, bool	`SUPERADMIN`, `MANAGER`, `USER`; inactive users can't log in
`ApiToken`		
`id`, `name`	uuid PK, text	
`prefix`, `tokenHash`	text, unique hash	First characters shown in the UI to recognise it; the hash of the full token (the token itself is never stored)
`role`, `createdById`	enum, FK → User	What the token may do; who made it
`lastUsedAt`, `revokedAt`	timestamptz, nullable	Usage; revocation
`AuditLog` : **append-only**		
`id`, `time`	bigint PK, timestamptz	
`actorType`, `actorId`, `actorName`	text	User or token, and who
`action`, `entity`, `entityId`	text	What was done to what
`before`, `after`	jsonb	The record before and after
`ip`, `auditCamera`	text, bool	Caller address; whether it touched an audit camera
Indexes: (entity, entityId, time), (time)		
`LoginLog` : **append-only**		

Column	Type	Meaning
`time`, `username`, `userId`	timestamptz, text, uuid	Username as typed; the user if it matched one
`result`	enum	`SUCCESS`, `WRONG_PASSWORD`, `DISABLED`, `LOCKED`, `RATE_LIMITED`, `LOGOUT`
`ip`, `userAgent`	text	
`DeadEvent` : **append-only**		
`streamId`, `payload`, `reason`	text	The Redis stream entry id, the raw message, why it couldn't be stored
`ExportJob` : **PDF reports**		
`id`, `params`	uuid PK, jsonb	The filters and period asked for
`status`	enum	`QUEUED`, `DONE`, `FAILED`, `EXPIRED`
`runAt`, `attempts`, `error`, `finishedAt`		Job state
`filePath`, `sizeBytes`	text, int	The PDF under `/data/exports`
`createdById`, `createdByName`	text	Owner; users see only their own jobs

Not in Postgres: live camera state, sessions, the event stream and job queues live in Redis (section 11). Files (snapshots, clips, recordings, models, PDFs) live in the `/data` volumes; the tables hold only their paths.

7. Use cases

Six use cases are defined in `packages/contracts/usecases/`. The id is the camera's `useCase`. Each folder has `usecase.json`, the schemas for events, intervals, settings and live values, `metrics.json`, and example payloads that `npm test` checks.

Use case	Events	Intervals	Metrics	Main settings
`cashier_presence` Cashier presence	`cashier_unattended`, `customer_waiting`	`desk_unattended`	`queue_length`, `clerk_count`	clerk zone, visitor zone, absence and waiting timeouts
`restaurant_counter` Restaurant occupancy	`capacity_alert`	`table_session`	`occupancy`, `occupied_tables`	dining zone, tripwire, shadow zone, max and warning capacity, optional table monitoring
`restaurant_table` Restaurant tables	none	`table_session`	`occupied_tables`, `table_occupancy_pct`	named table polygons, dwell time
`vehicle_gate` Vehicle gate	`vehicle_crossing`	none	none	tripwire, ROI polygon, tracker tuning, ANPR (Indonesian plates)
`horse_riding` Pony riding	`horse_crossing`	none	`active_horses`	tripwire, ROI polygon, tracker tuning, display labels
`system` System	`engine_started`, `events_dropped`, `config_rejected`, `model_validation_failed`, `disk_low`	`camera_offline`	none	none (not a camera use case)

Every camera use case also has the model settings (`model_id`, `imgsz`, `confidence_threshold`) and an optional `nx_bookmark`. Settings marked `x-minRole: SUPERADMIN` in the schema (model, tracker tuning) can only be changed by a superadmin; the API checks this on save.

Adding or changing a use case

1. Write the contract: `packages/contracts/usecases/<id>/` with its schemas and examples, then `npm run generate` and `npm test`.
2. Write the engine pipeline: `services/engine/usecases/<id>/pipeline.py`, one `BasePipeline` subclass, plus tests under `tests/`.
3. No migration and no dashboard change are needed for it to appear in the generic event, interval and metric views. A custom report page in the dashboard is optional.

> **Change rules.** Only add optional fields. Never rename, remove or change the meaning of a field. A breaking change bumps the use case's `schemaVersion`; old rows keep theirs and stay readable.

8. Users, roles and login

Three fixed roles, **SUPERADMIN > MANAGER > USER**. A superadmin is normally a Magnet engineer, a manager is the customer's admin and a user is a viewer. Hiding menu items in the UI is only a convenience; the API decides.

Action	USER	MANAGER	SUPERADMIN
Dashboards, live view, events, camera status and frames, CSV and PDF exports, reading site settings	yes	yes	yes
Zones, thresholds, enable or disable a camera, config history and rollback		yes	yes
Users (except superadmins), models list, system log, keeping an event's clip		yes	yes
Stream URLs, model, tracker tuning, engineGroup, use case, audit flag, recording settings			yes
Create or delete cameras, superadmin accounts, API tokens, model upload and jobs, editing site settings, debug view			yes

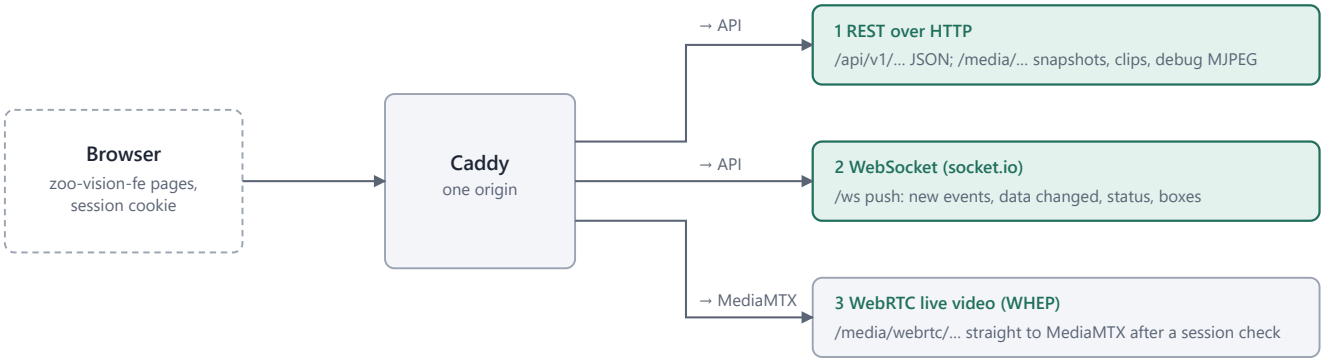
How login works

- `POST /api/v1/auth/login` sets a `zoo_session` cookie (HttpOnly, SameSite=Lax). The session lives in Redis and slides 12 hours from the last request, ending 7 days after login at the latest. Disabling a user or changing a role takes effect on their next request.
- Scripts, Postman and Swagger can use an **API token** instead: `Authorization: Bearer <token>`. Tokens are created by a superadmin, stored hashed, carry a role and can be revoked. The WebSocket accepts only the cookie.
- Protection: logins are limited to 10 a minute per address, and five wrong passwords in 15 minutes lock the account (`423 ACCOUNT_LOCKED`). Every attempt goes to the login history.
- The engine is a third kind of caller: it sends `ENGINE_TOKEN`, and only to `/internal/engine/*`.

> **Errors have one shape everywhere:** `{statusCode, code, message, errors?}`. Switch on `code` (stable) rather than `message`. The full list, with what the UI should do for each, is in `docs/fe-contract.md` section 4; the codes are defined in `apps/api/src/common/errors.ts`.

9. Connecting the frontend

The dashboard (`zoo-vision-fe`) reaches the backend in **three ways**, all through one address: the site that serves the dashboard itself. Caddy sends each path to the right service, so the frontend needs no backend URL, no CORS setup and no secret.



Green: answered by the NestJS API. Live video is the one path the API doesn't serve; it only answers Caddy's "is this browser logged in?" check (`GET /api/v1/auth/check`).

Way	Paths	Use it for	Described in
1 REST	`/api/v1/...`	Everything you read or change: login, cameras, config, reports, lists, exports, users, models, settings	`openapi.json` , `fe-contract.md` §3–10
	`/media/...`	Files used straight in HTML: `<img src={snapshotUrl}>` , `<video src={clipUrl}>` , the debug stream. Use the URLs the API gives you; don't build them	`fe-contract.md` §11
2 WebSocket	`/ws`	Being told that something changed, plus small live values (camera status, boxes). Never the source of truth	`fe-contract.md` §12, `index.d.ts`
3 WebRTC	`/media/webrtc/<path>/whep`	Live camera video. Take the URL from the camera's `liveUrls` ; it is `null` when there is no live video (file cameras, disabled cameras, demo stack)	`fe-contract.md` §11

Which page uses what

Page	REST	WebSocket	WebRTC / media
Login	`POST /auth/login` , then `GET /auth/me` for name and role	connect after login	
Dashboard	`/reports/*` , `/camera-status` , `/site-settings` (time zone)	`event.created` for toasts; `data.changed` → refetch	snapshots
Live	`/cameras` (for `liveUrls`), `/cameras/:id/config` (zones to draw)	room `status` : tiles and counters; room `camera:{id}` : boxes	WebRTC video

Page	REST	WebSocket	WebRTC / media
Events	`/events` with a cursor, `POST /events/:id/evidence`	`data.changed` → refetch	snapshots, clips
Reports	`/reports/events/stats`, `/reports/intervals/stats`, `/reports/metrics/series`, `/intervals`	`data.changed` → refetch	
Camera config and zone editor	`/cameras`, `/usecases` (settings schemas, what to draw), `/cameras/:id/frame` (background image), `POST /cameras/:id/config`, versions, rollback	room `status`: Saved → Applied when `appliedVersion` matches	frame JPEG
Exports	CSV links under `/exports/*.csv`; PDF: `POST /exports/jobs`, `GET /exports/jobs`, then the file		
Admin	`/users`, `/api-tokens`, `/models`, `/site-settings`, `/system-log/*`	`audit.notice`, `disk.warning` (managers)	debug MJPEG (superadmin)

Rules the frontend must follow

- **REST is the truth; the socket only nudges.** There is no replay: after every connect or reconnect, refetch what the page shows and subscribe to rooms again. Build tables from REST, not from `event.created`.
- **Only call the API from the browser.** The session is a cookie in the browser; the Next.js server has none.
- **Open the app through Caddy** in development (`http://localhost:8080`), not the Next.js port, or cookies and the socket won't work.
- **401 → login page. 403 → show the message** (and for `ROLE_REQUIRED`, disable the fields it names). A `session.ended` message means the same as a 401, except `role_changed`: refetch `/auth/me` and reconnect.
- **Switch on `code`, not `message`**, for every error.

How the frontend stays in sync with the backend

The backend publishes a **contract**: three files at a git tag `contract-vX.Y` on the backend repo. In the frontend, `npm run contract:pull <tag>` fetches them and runs orval, which generates a typed axios client from the OpenAPI file.

File	Gives the frontend
`apps/api/openapi.json`	Every REST endpoint with its parameters and response types → the generated hooks (`useCamerasList`, …)
`packages/contracts/generated/web/index.d.ts`	Types for every socket message (`ServerToClientEvents`, `ClientToServerEvents`) and each use case's settings, events, intervals and live values
`docs/fe-contract.md`	How it behaves: roles, error codes, paging, report parameters, media, socket rooms, and a changelog per tag

Until `v1.0`, a breaking change is allowed only with a changelog entry. From `v1.0`, changes to `/api/v1` are additive only.

10. API reference

All 58 operations in `apps/api/openapi.json`, grouped by area, plus the WebRTC route, which MediaMTX answers. Paths are under `/api/v1` unless shown in full. For the exact parameters and response shapes, open Swagger at `/api/docs` on a running API (set `SWAGGER=public` in development), or read the OpenAPI file. The minimum role is shown where it isn't USER.

Method	Path	What it does	Role
Health and auth			
GET	`/api/health`	Health check for Compose; 503 lists the failing dependency	public
POST	`/auth/login`	Log in; sets the session cookie	public
POST	`/auth/logout`	End this session	
GET	`/auth/check`	204 or 401, no body; Caddy's gate for live video	
GET	`/auth/me`	Who is calling: user or token, name, role	
PUT	`/auth/password`	Change own password; ends the user's other sessions	session only
Cameras and config			
GET	`/cameras`	List cameras; fields shown depend on the role	
POST	`/cameras`	Create a camera (also creates its MediaMTX path)	SUPERADMIN
GET	`/cameras/{id}`	One camera	
PATCH	`/cameras/{id}`	Edit; managers may change only name, area and enabled	MANAGER
DELETE	`/cameras/{id}`	Delete; refused with `CAMERA_HAS_DATA` once it has analytics	SUPERADMIN
GET	`/cameras/{id}/stream-info`	Is the stream arriving, which codec, recording disk cost	MANAGER
GET	`/cameras/{id}/config`	Active config version and its settings	
POST	`/cameras/{id}/config`	Save a new version (send `baseVersion`); the engine applies it live	MANAGER
GET	`/cameras/{id}/config/versions`	Version history	MANAGER
GET	`/cameras/{id}/config/versions/{version}`	One past version	MANAGER
POST	`/cameras/{id}/config/rollback`	Make an older version active (as a new version)	MANAGER
GET	`/usecases`	The use case registry: labels and settings schemas for the zone editor and forms	
Live state			
GET	`/camera-status`	Every camera: online (reported within 15 s), state, applied version, live counters	
GET	`/cameras/{id}/status`	The same for one camera	

Method	Path	What it does	Role
GET	`/cameras/{id}/frame`	Latest JPEG from the engine; 404 until it sends one	
Analytics: lists, reports, exports			
GET	`/events`	Event list, cursor paged, filterable	
GET	`/intervals`	Interval list, cursor paged	
GET	`/reports/events/stats`	Event counts by time bucket and grouping field	
GET	`/reports/intervals/stats`	Interval counts and durations by bucket	
GET	`/reports/metrics/series`	Metric time series (avg, max, …) by bucket	
GET	`/exports/events.csv`	Streamed CSV of events; audited	
GET	`/exports/intervals.csv`	Streamed CSV of intervals	
GET	`/exports/metrics.csv`	Streamed CSV of metrics	
GET	`/exports/summary.csv`	Per camera per hour or day summary	
POST	`/exports/jobs`	Start a PDF report job; 503 when Gotenberg isn't configured	
GET	`/exports/jobs`	Your own PDF jobs and their progress	
GET	`/exports/jobs/{id}/file`	Download a finished PDF	owner

Method	Path	What it does	Role
Media and evidence			
GET	`/media/snapshots/{path}`	An event snapshot JPEG	
GET	`/media/clips/{eventId}`	The event's clip (kept, or cut from recordings); `RECORDING_NOT_AVAILABLE` when gone	
GET	`/media/debug/{id}`	MJPEG of the engine's annotated frames	SUPERADMIN
POST	`/events/{id}/evidence`	Keep this event's clip past normal recording retention	MANAGER
POST	`/media/webrtc/<path>/whep`	Start WebRTC live video. Not in `openapi.json`: Caddy checks the session, then MediaMTX answers. Use the URL from the camera's `liveUrls`	
AI models			
GET	`/models`, `/models/{id}`	List and read registry entries	MANAGER
POST	`/models`	Upload an ONNX file (multipart); `.pt` is refused because it can run code when loaded	SUPERADMIN
POST	`/models/rescan`	Register files dropped into `models/inbox/` and the built-ins	SUPERADMIN
POST	`/models/{id}/validate`	Queue a test load and inference on the engine	SUPERADMIN
POST	`/models/{id}/convert`	Queue a `.pt` → ONNX conversion	SUPERADMIN

Method	Path	What it does	Role
DELETE	`/models/{id}`	Remove; refused with `MODEL_IN_USE` while a camera uses it	SUPERADMIN
Users, tokens, site			
GET	`/users`	List users	MANAGER
POST	`/users`	Create a user (a superadmin account needs SUPERADMIN)	MANAGER
PATCH	`/users/{id}`	Edit, disable, change role or reset password	MANAGER
GET	`/api-tokens`	List API tokens	SUPERADMIN
POST	`/api-tokens`	Create one; the only response that ever shows the raw token	SUPERADMIN
DELETE	`/api-tokens/{id}`	Revoke	SUPERADMIN
GET	`/site-settings`	Park name, time zone, retention days	
PATCH	`/site-settings`	Change them	SUPERADMIN
System log			
GET	`/system-log/audit` (+ `.csv`)	Who changed what, before and after	MANAGER
GET	`/system-log/logins` (+ `.csv`)	Login history and failures	MANAGER
GET	`/system-log/system-events` (+ `.csv`)	Engine starts, offline cameras, rejected config, low disk	MANAGER

Engine-only routes (not in Swagger, never routed by Caddy)

Method	Path	What it does
GET	`/internal/engine/cameras?group=<g>`	Enabled cameras of the group, with active settings (schema defaults filled in), decrypted source URL, target FPS and model
POST	`/internal/engine/models/{id}/validation`	The engine reports a validation result
POST	`/internal/engine/models/{id}/conversion`	The engine reports a conversion result

Each needs `Authorization: Bearer <ENGINE_TOKEN>`. A user session or API token gets 401.

11. WebSocket and engine channels

WebSocket for the browser: socket.io at `/ws`

WebSocket transport only, session cookie only, and the `Origin` must match the host. The message types are in `packages/contracts/ws/` and in the frontend's generated types file.

Message	Room	When
`event.created`	`alerts` (everyone)	Each new event, after the ingest commit
`data.changed`	`alerts`	At most every 5 s after commits: tells pages to refetch their data
`status.snapshot`	`status` (on request)	Every 2 s while anyone is in the room: all cameras' live status
`boxes`	`camera:{id}`	Up to 10 a second while anyone is in the room: `{t, sentAt, boxes}`, coordinates 0 to 1, track ids
`audit.notice`	`managers`	Someone changed an audit camera
`disk.warning`	`managers`	A data disk crossed 85% (warning) or 95% (critical), once per crossing
`session.ended`	`user:{id}`	The session was ended (logout elsewhere, disabled, role change); then disconnect
`subscribe` / `unsubscribe`	sent by the client	Join or leave `status` or a `camera:{id}` room

Redis: the contract between engine and API

Key or channel	Direction	Purpose
stream `events`	engine → API	Durable events, interval open and close, metric samples
pub/sub `cfg:changed`	API → engine	"Camera X has a new config version"; the engine then refetches its cameras
hash `cam:{id}:status` (TTL ~15 s)	engine → API	Heartbeat every 5 s: fps, state, applied version, error, `live` counters. If it expires, the camera is offline
key `cam:{id}:frame`	engine → API	Latest raw JPEG every 2 s, for the zone editor and grid thumbnails
pub/sub `cam:{id}:boxes`, key `cam:{id}:active`	both ways	Per-frame boxes for the live overlay. The API keeps the `active` key alive while a browser is in the camera's room; only then does the engine publish boxes
key `cam:{id}:debug`, pub/sub `cam:{id}:debugframes`	both ways	While the API keeps the key alive, the engine publishes annotated JPEGs for the debug view
list `model:jobs`	API → engine	Model validation and conversion jobs

The `live` values each pipeline puts in its status are typed per use case by `packages/contracts/usecases/<id>/live.schema.json`. The engine test `tests/test_live_contract.py` catches a renamed field before the dashboard does.

The frontend contract

The dashboard team builds against three files, pinned together by a git tag `contract-vX.Y`: `apps/api/openapi.json` (they generate an axios client from it with orval), `packages/contracts/generated/web/index.d.ts` (WebSocket and use

case types) and `docs/fe-contract.md` (the behaviour behind the endpoints, with a changelog per tag). `openapi.json` is generated by `npm run openapi` and CI fails if it is stale.

12. Running it

Three ways to run it

Goal	How	Guide
Frontend work, no GPU	Compose profile `demo`: fake cameras, users and two weeks of history from `demo-seed` and `demo-feed`	`docs/fe-dev-setup.md`
Backend work on a laptop	Postgres, Redis and the API in Compose; the engine from source with `uv run python main.py`; MediaMTX in Compose	`docs/10_LOCAL_QUICKSTART.md`
A park server	Everything in Compose with profiles `engine` and `web`, on a GPU server with the NVIDIA Container Toolkit	`deploy/README.md`

Compose services and profiles (`deploy/docker-compose.yml`)

Service	Profile	Role
`postgres`	always	Postgres 17; creates the `zoo_api` role on first start. No host port
`redis`	always	Redis 7.4 with a password and AOF, on `127.0.0.1` for the host-networked engine
`api-migrate`	always	One-shot `prisma migrate deploy` as `zoo_owner`, then exits
`api`	always	The NestJS API on `127.0.0.1:${ZOO_API_PORT}`
`caddy`	always	The only public ports; modes `dev`, `internal-tls` (default) or `custom-tls`
`gotenberg`	always	PDF rendering, internal only
`mediamtx`, `zoo-ai-engine`	`engine`	Video and the Python engine (needs the GPU)
`web`	`web`	The dashboard, built from the `zoo-vision-fe` checkout
`demo-seed`, `demo-feed`	`demo`	Fake data for frontend development

First run, in short

```
cp deploy/.env.example deploy/.env          # fill in the secrets (deploy/README.md, "Secrets")
cd deploy && docker compose up -d --build   # add COMPOSE_PROFILES=engine,web on a server
docker compose exec api node dist/cli.js create-superadmin --username admin --name "Admin"
                                            # with ZOO_ADMIN_PASSWORD set in the environment
python tools/fetch_models.py                # model weights are not in git
```

Then log in through Caddy (not straight to the API port), add cameras in the UI (or import a YAML list once with `tools/import_yaml_config.py`), and draw their zones.

Secrets and the variables that matter most

Variable	Meaning
`ENGINE_TOKEN`	One value shared by API and engine, at least 32 characters
`CAMERA_URL_KEY`	Encrypts stored camera URLs, which contain passwords (base64 of 32 bytes)

Variable	Meaning
`POSTGRES_PASSWORD`, `ZOO_API_DB_PASSWORD`, `REDIS_PASSWORD`	Database and Redis passwords (keep them URL-safe)
`SWAGGER` / `ZOO_SWAGGER`	`off` (default), `superadmin`, or `public` (development only)
`COOKIE_SECURE`	`false` only for plain HTTP in development
`ENGINE_GROUP`, `DEVICE`	Engine: which cameras this process runs; `cpu` or `cuda:0`

Tests

Part	Command	Notes
Engine	`cd services/engine && uv run pytest`, `uv run ruff check .`	Pipelines are tested with synthetic detections; no model needed
API	`cd apps/api && npm test`, `npm run test:e2e`	e2e needs Docker: Testcontainers starts Postgres and Redis
Contracts	`cd packages/contracts && npm test && npm run check`	Checks the examples and that generated code is current
One pipeline on a video	`uv run python test_video.py --video <file> --pipeline <name>`	Uses a rule YAML in `configs/rules/`, not the API

13. Repo map and where to read more

Folder	What it holds
`services/engine/`	Python engine: `core/` (supervisor, worker, streams, model pool, Redis I/O), `usecases/` (one pipeline each), `contracts/` (generated), `tests/`
`apps/api/`	NestJS API: `src/<module>/`, `prisma/` (schema and migrations), `test/` (e2e), `openapi.json`
`packages/contracts/`	JSON Schemas and their generated TypeScript and Python
`deploy/`	Compose file, Caddyfile, `mediamtx.yml`, `.env.example`
`tools/`	`fetch_models.py`, `export_model.py`, `pick_coordinates.py`, `import_yaml_config.py`
`integrations/nx/plugin/`	The frozen C++ Nx plugin from the prototype
`docs/`	Guides, ADRs, the design spec and phase plans

Reading order

#	Document	Read it for
1	`README.md`	Repo map and quick-start commands
2	`docs/superpowers/specs/2026-10-08-zoo-vision-production-architecture-design.md`	The full design and the reasons for it. Start with sections 1, 3 and 4
3	`docs/adr/README.md` and ADR-001 to ADR-008	Individual decisions; ADR-008 explains the move away from Nx
4	`services/engine/README.md`	Engine setup, environment, adding a use case
5	`apps/api/README.md`	API modules, environment, migrations and grants, engine routes
6	`packages/contracts/README.md`	Schema layout and change rules
7	`docs/fe-contract.md`	API behaviour in detail: errors, paging, reports, media, WebSocket
8	`docs/10_LOCAL_QUICKSTART.md`, `docs/fe-dev-setup.md`, `deploy/README.md`	Running it on a laptop, for the frontend, or on a server
9	`docs/superpowers/specs/` Phase 3, 4a and 4b designs	Details of one feature area when you need them
10	`docs/02`, `03`, `05`, `08`, `09`	Model training, hardware sizing, model export, testing with video, MediaMTX

> **Historical, read with care:** `docs/01_IMPLEMENTATION_PLAN.md`, `docs/06_MAGNET_NX_PLUGIN_IMPLEMENTATION_PLAN.md` and `docs/architecture_best_practice_recommendation.md` describe the prototype, which was built around Nx Witness. The production design replaced that. Parts of `docs/07_INSTALLATION_GUIDE.md` may also date from before the rebuild.

14. Glossary

Term	Meaning
Use case	A kind of analysis a camera runs (cashier presence, vehicle gate, …). Defined by its contract; implemented by one engine pipeline
Pipeline	The engine class for one use case: takes detections and tracks, applies zones and timers, produces messages and live values
Contract	The JSON Schemas in `packages/contracts` that every part generates its types from
Config version	One saved set of a camera's settings. Versions only increase; rollback makes a new one
Event / Interval / Metric	The three shapes of analytics data: a moment, a span of time, a number per minute
Envelope	The common wrapper of every stream message (kind, camera, time, config version, schema version, data)
Engine group	The set of cameras one engine process runs (`ENGINE_GROUP`)
Zone, ROI, tripwire	Polygons and lines drawn on the camera image, in 0 to 1 coordinates. A tripwire counts crossings; an ROI limits where detection counts
Audit camera	A camera whose config changes are flagged and notified and whose evidence is kept longer, because moving a tripwire is the easiest way to hide sales
Evidence / kept clip	An event's video clip copied out of the continuous recording so it outlives the normal retention
Dead event	A stream message that could not be stored, parked in `DeadEvent` so it doesn't block the rest
WHEP	The WebRTC playback protocol MediaMTX uses for live video in the browser
ANPR	Automatic number plate recognition, used at the vehicle gate (ADR-005)
ByteTrack	The multi-object tracker that gives each detection a stable id across frames (ADR-007)
`x-minRole`	A schema annotation on a settings field: the lowest role allowed to change it